

useContext

- React's useContext hook allows developers to access and share state across the entire component tree without passing props down manually at every level.
- Introduced in React 16.8, useContext works alongside the Context API, which was introduced earlier in React 16.3.
- While the Context API itself allows you to create a global state that can be accessed by any component, useContext simplifies the way this state is consumed.
- When you create a context using React.createContext(), you can think of it as creating a global state container.
- Components that need access to this state can subscribe to it using the useContext hook.
- This approach eliminates the need for "prop drilling," where you pass data through multiple layers of components, making your code cleaner and more maintainable.

Prop Drilling

- Prop drilling, also known as "threading props" or "component chaining," refers to the process of passing data from a parent component down to nested child components through props.
- Prop drilling occurs when a prop needs to be passed through several layers of nested components to reach a deeply nested child component that actually needs the prop.
- Each intermediary component in the hierarchy has to pass the prop down, even if it doesn't use the prop itself.
- Consider a scenario where you have a top-level component that fetches data from an API and needs to pass this data down to multiple nested child components.
- Instead of directly passing the data to each child component, you pass it through each intermediary component in the hierarchy until it reaches the desired child component.
- This passing of props through multiple levels of components is what prop drilling entails.

Example:

Let's consider a scenario where a User component needs access to a user object stored in App, but it's deeply nested inside a Profile component.

App.jsx

```
import React from "react"
import Dashboard from "../useContext/PropsDrilling/Dashboard"

function App() {
  const user = {
    name: "Cecilia",
    email: "cecilia@gmail.com"
  }
  return (
    <div>
      <Dashboard user={user}/>
    </div>
  )
}
export default App
```

Dashboard.jsx

```
import React from 'react'
import Profile from './Profile'

function Dashboard({user}) {
  return (
    <div>
      <Profile user={user}/>
    </div>
  )
}
export default Dashboard
```

Profile.jsx

```
import React from 'react'
```

```
import UserDetails from './UserDetails'
```

```
function Profile({user}) {  
  return (  
    <div>  
      <UserDetails user={user}/>  
    </div>  
  )  
}  
export default Profile
```

UserDetails.jsx

```
import React from 'react'  
function UserDetails({user}) {  
  return (  
    <div>  
      <h1>Name: {user.name}</h1>  
      <h1>Email: {user.email}</h1>  
    </div>  
  )  
}  
export default UserDetails
```

- App has user state but needs to pass it through Dashboard and Profile, even though they don't use it.
- If UserDetails was deeper in the component tree, the problem would get worse.

Why is Prop Drilling a Problem?

1. Unnecessary Prop Passing – Components that don't need the prop still have to pass it down.
2. Hard to Maintain – As the application grows, it becomes difficult to track where props are being passed.

3. Code Clutter – Too many props in intermediate components make code harder to read and debug.
4. Difficult Updates – If a prop structure changes, multiple components need to be updated.

Using useContext to Avoid Prop Drilling

UserContext.jsx

- This component creates and exports a context (UserContext), a provider (UserProvider), and a custom hook (useUser).

```
import { createContext, useContext } from "react";
const UserContext = createContext();

const UserProvider = ({ children }) => {
  const user = { name: "Cecilia", email: "cecilia@gmail.com" };
  return
  <UserContext.Provider value={user}>
    {children}
  </UserContext.Provider>;
};

const useUser = () => useContext(UserContext);

export { useUser };
export default UserProvider;
```

Importing createContext and useContext

- createContext() is used to create UserContext, which will store user information.
- useContext() is used later to retrieve data from UserContext.

Creating UserContext

- const UserContext = createContext();
- This creates a new context that can be used throughout the app to store and access user-related data.

Defining UserProvider

- UserProvider is a component that wraps parts of the application that need access to the user data.
- It stores a user object (name and email) and provides it to child components through `<UserContext.Provider>`.

Creating useUser() Hook

- Instead of calling `useContext(UserContext)` everywhere, we define `useUser()`, which returns the current value of `UserContext`.

Exporting Components

- UserProvider is exported as default so it can be imported without curly braces.
- `useUser` is exported as a named export, so it must be imported with `{ useUser }`.

User.jsx

This component uses the `useUser` hook to get user details and display them.

```
import { useUser } from "../UserContext";

const User = () => {
  const { name, email } = useUser(); // Access user data from context
  return (
    <div>
      <h2>User Details</h2>
      <p><strong>Name:</strong> {name}</p>
      <p><strong>Email:</strong> {email}</p>
    </div>
  );
};

export default User;
```

Importing useUser

- The `useUser` hook is imported to retrieve user data from `UserContext`.

Retrieving User Data

- `const { name, email } = useUser()`

- This extracts the name and email values stored in context.

Displaying User Information

- A `<div>` contains:
 - `<h2>` heading: "User Details".
 - Two `<p>` elements that dynamically display name and email values from context.

Exporting the Component

- `export default User;`
- This makes User available for use in other files (like App.jsx).

App.jsx

- This component wraps the application with `UserProvider` so that the User component can access the context.

```
import React from "react";
```

```
import User from "../components/User";
```

```
import UserProvider from "../components/UserContext";
```

```
function App() {  
  return (  
    <UserProvider>  
      <div>  
        <h1>Welcome to the User Profile App</h1>  
        <User />  
      </div>  
    </UserProvider>  
  );  
}
```

```
export default App;
```

Importing Components

- `UserProvider` (from `UserContext.jsx`) wraps the entire app.

- User (from User.jsx) is used to display user details.

Wrapping with UserProvider

- <UserProvider> ensures that all child components (like User) can access context.
- Without this wrapper, User would not receive any user data.

Displaying Components

- <h1> welcomes users.
- <User /> displays user details.

Exporting App

- export default App; allows App.jsx to be used as the main entry point.

Example 2

1. Creating a ThemeContext to store the theme state.
2. Providing the Theme Context to the entire app.
3. Consuming the Theme Context to toggle and display the theme.

ThemeContext.jsx

- This component creates the context and provides theme data.
- ```
import { createContext, useContext, useState } from "react";
const ThemeContext = createContext();
const ThemeProvider = ({ children }) => {
 const [theme, setTheme] = useState("light");
 const toggleTheme = () => {
 setTheme((prevTheme) => (prevTheme === "light" ? "dark" : "light"));
 };

 return (
 <ThemeContext.Provider value={{ theme, toggleTheme }}>
 {children}
 </ThemeContext.Provider>
);
};
```

```
const useTheme = () => useContext(ThemeContext);
```

```
export { useTheme };
```

```
export default ThemeProvider;
```

- createContext() - Creates a ThemeContext to store the theme state.
- ThemeProvider Component - Manages theme state (light/dark) and provides a toggleTheme function.
- useTheme Hook - Allows child components to access theme data (theme & toggleTheme).

### **ThemeToggle.jsx**

```
import { useTheme } from "../ThemeContext";
```

```
const ThemeToggle = () => {
```

```
 const { theme, toggleTheme } = useTheme(); // Access theme and toggle function
```

```
 return (
```

```
 <div style={{ textAlign: "center", padding: "20px" }}>
```

```
 <h2>Current Theme: {theme}</h2>
```

```
 <button onClick={toggleTheme}>
```

```
 Switch to {theme === "light" ? "Dark" : "Light"} Mode
```

```
 </button>
```

```
 </div>
```

```
);
```

```
};
```

```
export default ThemeToggle;
```

- useTheme() is used to access the theme state and the toggleTheme function.
- The button toggles the theme when clicked.



### **App.jsx**

```
import React from "react"
import User from "../components/User"
import UserProvider from "../components/UserContext"
import ThemeProvider from "../components/ThemeContext";
import ThemeToggle from "../components/ThemeToggle";

function App() {

 return (
 <>
 <ThemeProvider>
 <div>
 <h1>Theme Switcher App</h1>
 <ThemeToggle />
 </div>
 </ThemeProvider>
 </>
)
}

export default App
```

- ThemeProvider wraps the entire app.
- ThemeToggle allows users to switch between light and dark modes.

### **Benefits of Using useContext**

- Simplified Component Structure: By leveraging useContext, you can avoid passing props through intermediate components, leading to a cleaner and more concise component tree.

- Improved Code Readability: Context providers and consumers are closely related, making it easier to understand how data flows through your application.
- Easier Maintenance: With useContext, you reduce the likelihood of prop drilling-related bugs and make your codebase more maintainable in the long run.

For more practice -

<https://medium.com/zestgeek/mastering-reacts-usecontext-hook-simplifying-state-management-65894e6dc431>